CENTRO UNIVERSITÁRIO SALESIANO DE SÃO PAULO
UNISAL – CAMPUS MARIA AUXILIADORA

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Americana – SP
2026

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Trabalho de Conclusão de Curso apresentado como exigência parcial para a obtenção do grau de Bacharel em Engenharia da Computação no Centro Universitário Salesiano de São Paulo.
Orientador: Prof. Dr. Leonardo Grando.

Americana – SP
2026

Matheus Tomazella
Rafael Cesar Vanzo
Samuel dos Santos Carneiro

NOVADMIN: Módulo de Orçamento para Sistema ERP

Trabalho de Conclusão de Curso apresentado como exigência parcial para a obtenção do grau de Bacharel em Engenharia da Computação no Centro Universitário Salesiano de São Paulo.
Orientador: Prof. Dr. Leonardo Grando.

Trabalho de Conclusão de Curso defendido e aprovado em ___/11/2026, pela comissão julgadora:

Prof. Dr. Leonardo Grando, UNISAL

Americana – SP
2026

RESUMO

Apresenta a implementação de um módulo de orçamento, construído com o objetivo de centralizar dados relevantes e automatizar etapas de cálculo e a geração do documento de proposta, reduzindo o esforço necessário e aumentando a consistência do processo. O módulo foi desenvolvido com uma arquitetura em camadas, utilizando React para a interface web, Node.js com TypeScript e Express.js para a API REST, e PostgreSQL para o armazenamento de dados em um processo iterativo e incremental, com ênfase na validação contínua junto aos usuários finais e na entrega progressiva de funcionalidades. A interface organiza o processo de orçamentação em etapas sequenciais, facilitando o trabalho do usuário e garantindo flexibilidade para atender a situações reais do domínio. O resultado é um sistema que espelha fielmente o fluxo de trabalho da empresa, incorporando suas nomenclaturas, etapas e regras de negócio específicas, eliminando a necessidade de adequações operacionais forçadas.

Palavras-chave: arquitetura de software; orçamento; ERP.

ABSTRACT

Presents the implementation of a module for quotes, built to centralize relevant data and automate calculation steps and the generation of the proposal document, reducing the necessary effort and increasing the consistency of the process. The module was developed with a layered architecture, using React for the web interface, Node.js with TypeScript and Express.js for the REST API, and PostgreSQL for data storage in an iterative and incremental process, with emphasis on continuous validation with end users and progressive delivery of features. The interface organizes the quoting process into sequential steps, facilitating the user's work and ensuring flexibility to meet real-world situations in the domain. The result is a system that faithfully mirrors the company's workflow, incorporating its nomenclature, steps, and specific business rules, eliminating the need for forced operational adjustments.

Key-words: software architecture; quoting; ERP.

SUMÁRIO

1 INTRODUÇÃO 6
1.1 Contexto Empresarial 6
1.2 Objetivos 6
2 DESENVOLVIMENTO 7
2.1 Metodologia 7
2.2 Fluxo de Trabalho do Usuário 8
2.3 Tecnologias e Infraestrutura 11
2.3.1 Infraestrutura e Implantação 13
2.4 Resultados 14
2.4.1 Interface do Usuário 14
3 CONCLUSÃO 22

REFERÊNCIAS 23

GLOSSÁRIO 24

1 INTRODUÇÃO

O processo de elaboração de orçamentos constitui atividade recorrente em empresas de engenharia e construção. Envolvendo a coleta e consolidação de informações sobre clientes, obras, materiais e regras de precificação, é um processo complexo que tem potenciais riscos significativos de inconsistência e retrabalho, podendo gerar danos à reputação e ao lucro da empresa em casos extremos.

Esse trabalho descreve a extensão de um sistema previamente construído com a implementação de um módulo especializado para geração de orçamentos e propostas de serviço. O objetivo geral é centralizar dados relevantes e automatizar etapas de cálculo e a geração do documento de proposta, reduzindo o esforço necessário e aumentando a consistência do processo.

A interface segue o molde do sistema previamente implantado, utilizando uma plataforma web clara, com edição organizada em etapas sequenciais e suporte a colaboração controlada quando aplicável.

1.1 Contexto Empresarial

O cliente é uma empresa especializada na fabricação de moldes reutilizáveis de concreto para o setor de engenharia civil. Historicamente, o processo de elaboração de propostas comerciais era executado por meio de planilhas eletrônicas e procedimentos manuais, o que gerava riscos significativos de inconsistência, dificuldade de rastreamento de decisões e reduzida capacidade de automação.

Reconhecendo essas limitações operacionais, a empresa decidiu ampliar seu sistema corporativo NOVADMIN (já em uso para outras funcionalidades) por meio da implementação de um módulo especializado para orçamentação. Esta expansão permite consolidar dados dispersos, automatizar etapas críticas e estabelecer um ponto único de controle e auditoria para o processo de proposta.

Dessa forma, diferentemente de soluções de mercado que, por mais configuráveis que sejam, impõem uma lógica genérica à qual a empresa precisa se adaptar, o módulo de orçamentação do sistema espelha fielmente o fluxo de trabalho da empresa, incorporando suas nomenclaturas, etapas e regras de negócio específicas. Essa aderência ao processo interno elimina a necessidade de adequações operacionais forçadas e garante que a ferramenta sirva à empresa, e não o contrário.

1.2 Objetivos

Os principais objetivos deste projeto são: expandir o banco de dados para suportar as entidades de fornecedor, material, orçamento, escopo, elemento, custo fixo e opção de precificação; adicionar as funcionalidades necessárias para gerenciar esses dados; automatizar o cálculo dos custos do serviço a partir das informações inseridas; automatizar a

geração do documento de proposta de serviço; e implementar um sistema de versionamento para os orçamentos.

Além disso, os objetivos não funcionais incluem oferecer uma interface simples e intuitiva, manter o estilo previamente estabelecido para o sistema, garantir flexibilidade para que o usuário não seja limitado em suas operações, priorizar a automação dos processos para reduzir a possibilidade de erro humano e aumentar a eficiência do sistema, e permitir que múltiplos usuários editem partes de um registro simultaneamente.

2 DESENVOLVIMENTO

Essa seção descreve a metodologia adotada para a implementação do projeto, o fluxo de trabalho do usuário quando visto isolado da tecnologia e detalhes sobre as decisões técnicas arquiteturais e de infraestrutura que foram tomadas para atender aos requisitos do sistema.

2.1 Metodologia

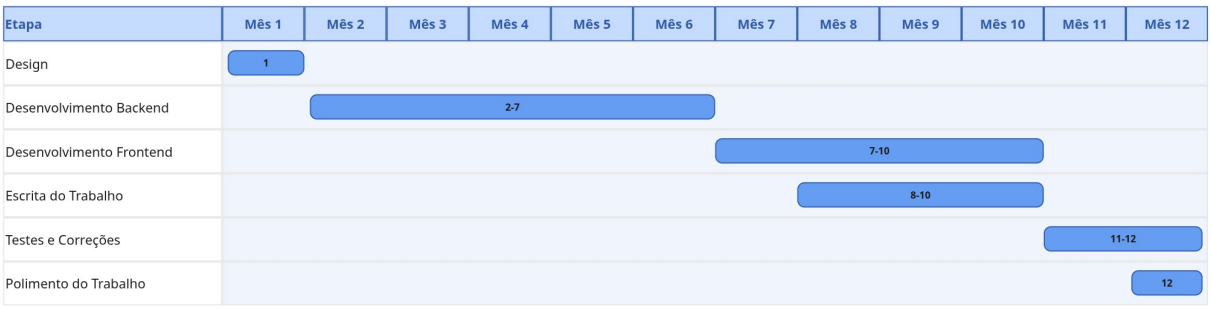
O desenvolvimento do módulo foi baseado em *frameworks* ágeis, seguindo uma abordagem iterativa e incremental, com ênfase na validação contínua junto aos usuários finais e na entrega progressiva de funcionalidades.

Os requisitos funcionais foram levantados por meio de sessões de trabalho com a equipe operacional da empresa, garantindo que as necessidades reais do domínio fossem compreendidas e incorporadas no *design*. Durante as fases de implementação, testes unitários foram aplicados sistematicamente para validar a lógica de negócio, especialmente nas operações críticas de cálculo dos custos, utilizando dados gerados pelas planilhas substituídas pelo sistema como garantia de que os resultados continuariam condizentes com o processo.

Aspectos como a alta flexibilidade requerida pelo domínio do sistema (que demanda suporte a múltiplas exceções de regras) e a necessidade de compreender o contexto específico do cliente foram gerenciados por meio de ciclos de avaliação contínua, permitindo refinar a solução conforme o aprendizado progrediu.

A Figura 1 apresenta o cronograma do projeto, que foi organizado em etapas complementares. A fase inicial de *design* teve duração de um mês, seguida por cinco meses de desenvolvimento do *backend* e quatro meses de desenvolvimento do *frontend*. O trabalho escrito foi produzido em paralelo às etapas de desenvolvimento, durante três meses. No dois meses finais, foram realizados simultaneamente os testes e correções e o polimento do documento.

Figura 1 – Cronograma de desenvolvimento do projeto



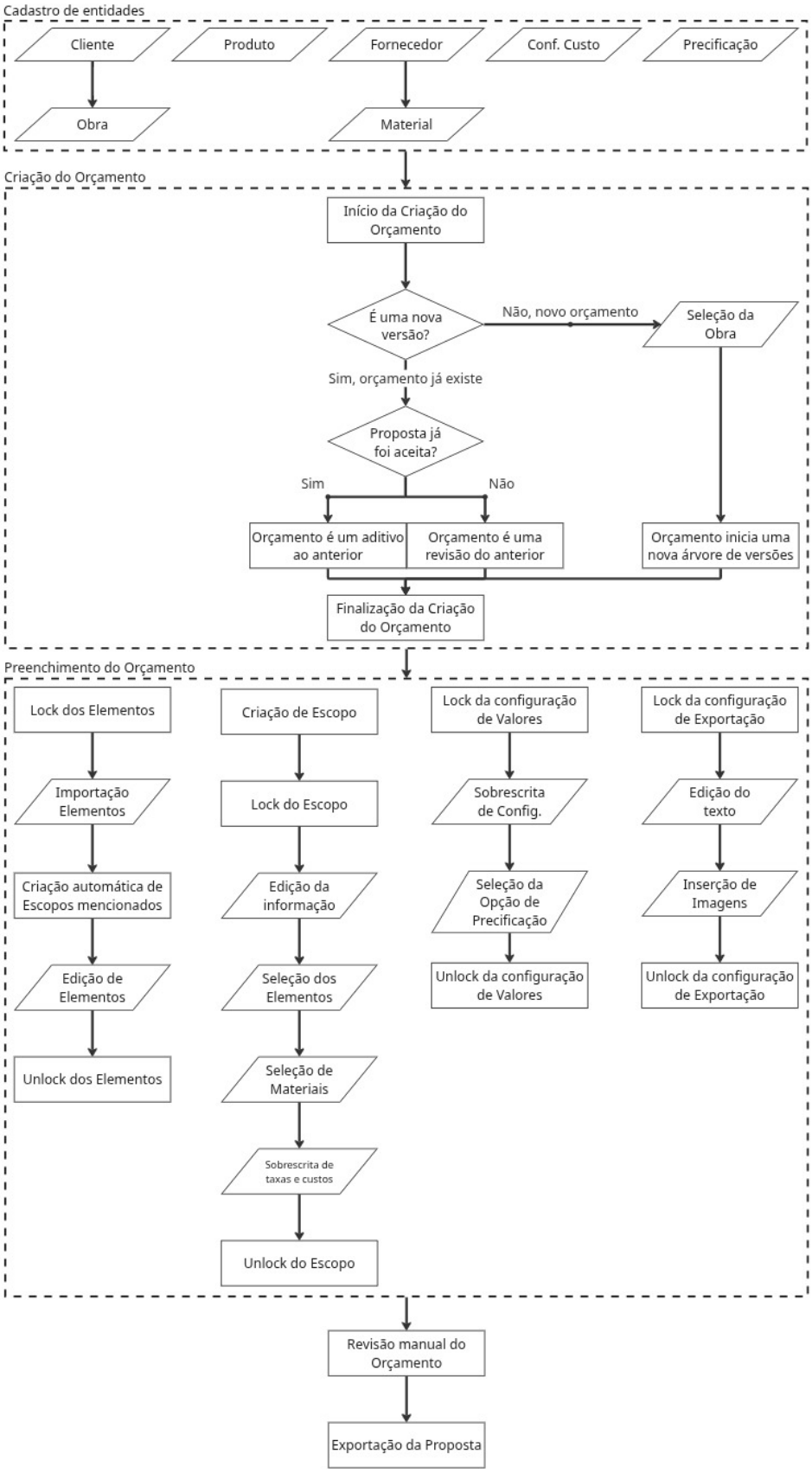
Fonte: Elaborado pelos autores, 2026.

2.2 Fluxo de Trabalho do Usuário

Esta seção descreve o fluxo de trabalho do módulo de forma abstrata, apresentando o caminho percorrido pelo usuário para elaborar um orçamento de maneira isolada, ou seja, sem considerar detalhes de implementação, tecnologias empregadas ou a aparência das telas. Esses aspectos são tratados nas seções seguintes.

A Figura 2 demonstra o fluxo completo, desde a inserção das entidades que compõem o Orçamento até a exportação do documento da proposta.

Figura 2 – Diagrama de fluxo de trabalho do módulo de orçamento



Fonte: Elaborado pelos autores, 2026.

O fluxo se inicia com a inserção das entidades que compõem o Orçamento: Cliente, Obra, Produto, Fornecedor, Material, Custos Fixos e Opções de Precificação. Todas são entidades simples, sem relacionamentos em sua composição, exceto por Material, que deve ser associado a um Fornecedor e Obra que deve ser associada a um Cliente.

Em seguida, o Orçamento é criado, sendo associado a um Cliente e a uma Obra, podendo também ser uma nova versão de um Orçamento anterior. Uma nova versão mantém relação com o Orçamento de origem e pode ser de dois tipos: uma revisão, caso o Orçamento ainda esteja aberto, ou um aditivo, caso já tenha sido concluído.

Posteriormente, ocorre a etapa de gerenciamento dos Elementos, na qual o usuário pode realizar a importação por meio de arquivo, bem como efetuar edições manuais. Durante esse processo, o gerenciamento de Elementos permanece indisponível para outros usuários. A inserção de Elementos por um arquivo também permite ao usuário criar automaticamente os Escopos correspondentes.

A seguir, pode ser feito o gerenciamento dos Escopos, uma entidade que representa um trecho do projeto e contém Elementos e configurações. Aqui o usuário também associa os materiais aos elementos e pode sobrescrever valores de cálculo para o contexto do Escopo. A fim de evitar conflitos de edição, o acesso a cada Escopo é restrito a um único usuário enquanto este estiver em uso.

A etapa seguinte envolve o gerenciamento de configurações de precificação, como sobrescrever valores de cálculo para o contexto do Orçamento, e selecionar a opção de precificação que será utilizada para o cálculo do valor final.

Por fim, ocorre a configuração do documento, etapa que permite ao usuário ajustar textos e formatação, inserir imagens e configurar a ordenação de elementos. Aqui o usuário também deve conferir manualmente as informações e valores finais calculados. Feito isso, a proposta pode ser exportada em formato PDF ou DOCX, finalizando o processo.

Um elemento importante desse processo é a inclusão do sistema de sobrescrita de valores para os escopos de cálculo. Configurações como a definição dos produtos, preços de materiais e taxas de consumo de matéria-prima para determinado tipo de elemento são realizadas fora de um orçamento e servem como fonte geral, porém todas podem ser sobrescritas tanto para um determinado Orçamento quanto para um Escopo naquele Orçamento.

Isso permite que o sistema se adapte a situações reais, que não necessariamente seguem o fluxo padrão, mas ainda são de grande importância, como a aplicação de descontos ou situações incomuns como falta de funcionários ou materiais. Essa flexibilidade é um dos fatores que tornam o sistema um substituto real ao processo anterior, garantindo que se adeque à empresa e não o contrário.

2.3 Tecnologias e Infraestrutura

O módulo de orçamento foi organizado em camadas para separar a interface, as regras de negócio e a persistência dos dados. Essa divisão evita que a tela dependa diretamente da forma como o banco está estruturado e facilita a manutenção quando alguma parte do processo precisa ser ajustada.

A camada de apresentação consiste em uma interface web desenvolvida em React (Meta Platforms, Inc., s.d.), escolha realizada pela necessidade de uma navegação responsiva entre listas, formulários e abas de edição, para a qual o React oferece abstrações adequadas. Essa tecnologia também já era utilizada para o sistema já implantado, permitindo uma integração simples.

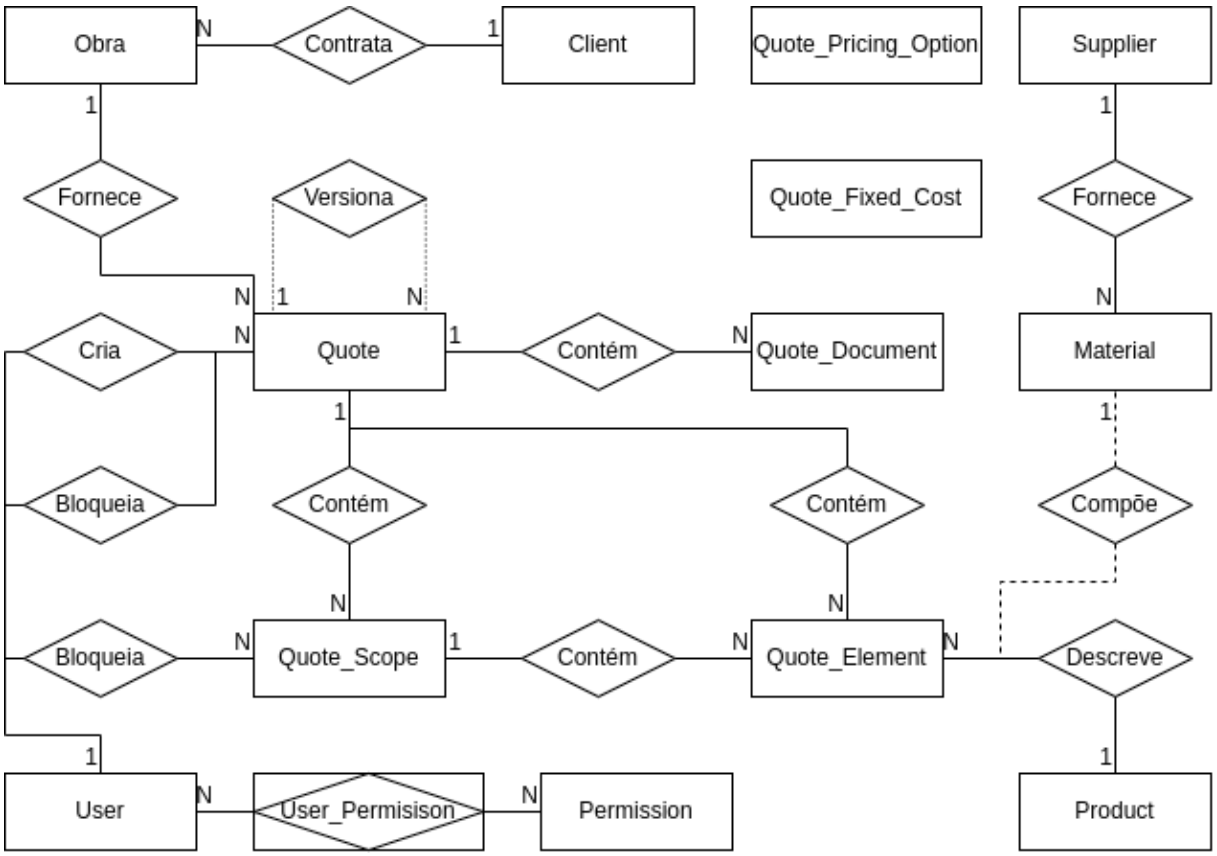
A camada de lógica de negócios consiste em uma API (Application Programming Interface) REST (Representational State Transfer) (FIELDING, 2000) implementada em Node.js (Node.js Foundation, s.d.) com TypeScript (Microsoft, s.d.) e Express.js (ExpressJS, s.d.). A API concentra as regras de domínio do módulo e expõe operações de criação, leitura, atualização, remoção e bloqueio de registros. A separação entre camadas foi adotada para reduzir acoplamento, facilitar evolução independente e centralizar regras de negócio em um único ponto de controle.

Uma decisão arquitetural relevante foi a utilização de uma biblioteca compartilhada entre essas camadas, contendo regras de domínio e tipos de dados relevantes. Essa abordagem reduz duplicação de código, principalmente em cálculos, transformações de dados e tipos. Consequentemente, garante uniformidade de comportamento entre as camadas e reduz a probabilidade de divergências entre interface e processos internos.

Para armazenamento de dados foi adotado PostgreSQL (PostgreSQL Global Development Group, s.d.), um sistema gerenciador relacional que oferece suporte adequado aos requisitos de integridade referencial e transacional demandados pelo domínio. O modelo relacional favorece a preservação de consistência e facilita consultas agregadas sobre múltiplas entidades.

A Figura 3 apresenta o esquema definido para atender às necessidades do sistema, representado por meio de um diagrama MER (Modelo Entidade-Relacionamento) (CHEN, 1976).

Figura 3 – Diagrama MER do banco de dados



Fonte: Elaborado pelos autores, 2026.

A análise da estrutura começa pelas entidades que representam elementos mais simples: clientes (*Client*), produtos (*Product*), obras, fornecedores (*Supplier*), materiais (*Material*), usuários (*User*), opções de precificação (*Quote_Pricing_Option*) e custos fixos (*Quote_Fixed_Cost*) que possuem poucas interações entre si e no geral servem apenas para guardar dados que serão úteis para outros elementos mais complexos.

Os elementos do orçamento são armazenados na tabela Quote_Element, cujos registros se relacionam com a tabela Quote_Scope, responsável por armazenar informações dos escopos do projeto, como dados de identificação e configuração. Por sua vez, o escopo está associado à tabela Quote, que representa o Orçamento.

A tabela Quote_Document também se relaciona com o Orçamento e armazena os anexos utilizados para a criação do documento de proposta, como imagens dos projetos referenciados.

Adicionalmente, a tabela Permission armazena as permissões do sistema e se relaciona com os usuários por meio da tabela intermediária User_Permission, sendo essencial para o controle de acesso.

Essa organização permite que o sistema trate suas entidades de forma independente, facilitando a implementação de sistemas como o bloqueio de edição necessário para a edição concorrente de itens em um mesmo Orçamento.

Assim, a arquitetura foi concebida para favorecer evolução incremental. Esta abordagem reduz riscos técnicos e viabiliza entrega progressiva de funcionalidades.

2.3.1 Infraestrutura e Implantação

Para facilitar desenvolvimento, testes e implantação, o projeto adota contêineres Docker (Docker, Inc., s.d.b) e orquestração via Docker Compose (Docker, Inc., s.d.a). A utilização de contêineres garante que o ambiente de execução seja consistente entre máquinas de desenvolvimento e servidores de produção, reduzindo inconsistências. A orquestração é empregada para montar serviços dependentes (API, banco de dados, serviços complementares) com configuração unificada, simplificando a criação de ambientes reproduzíveis e isolados.

O projeto adota um banco de dados relacional (SQL) devido à natureza fortemente relacional dos dados de domínio. O PostgreSQL foi selecionado pela robustez, maturidade de funcionalidades, comunidade ativa e suporte adequado em ambientes empresariais.

Para implantações em infraestrutura local, o projeto inclui scripts e configurações de serviço que automatizam inicialização e parada do sistema com integração ao estado do sistema operacional. Esta abordagem permite reinício automático do serviço, diminuindo o tempo que fica indisponível.

Além disso, para reduzir risco em *deploys* de produção, a infraestrutura foi montada de forma a suportar um fluxo de *release Blue/Green* (FOWLER, 2010). Nesse modelo, mantém-se dois ambientes idênticos, porém enquanto um está ativo e disponível aos usuários o outro recebe a nova versão, que é testada. Após validação, o roteamento é redirecionado para o ambiente atualizado, permitindo uma recuperação rápida ao manter a outra instância intacta caso algum problema ocorra. Com imagens Docker e orquestração, esse padrão é de simples implementação e combina bem com práticas de CI/CD para *deploys* controlados.

A infraestrutura também prevê centralização de logs, monitoramento básico e políticas de *backup/restore* para o PostgreSQL em múltiplos locais. A configuração por contêiner facilita a instrumentação e a padronização de métricas e alertas e o armazenamento de versões antigas do banco de dados garante recuperação em situações catastróficas com perda mínima de trabalho.

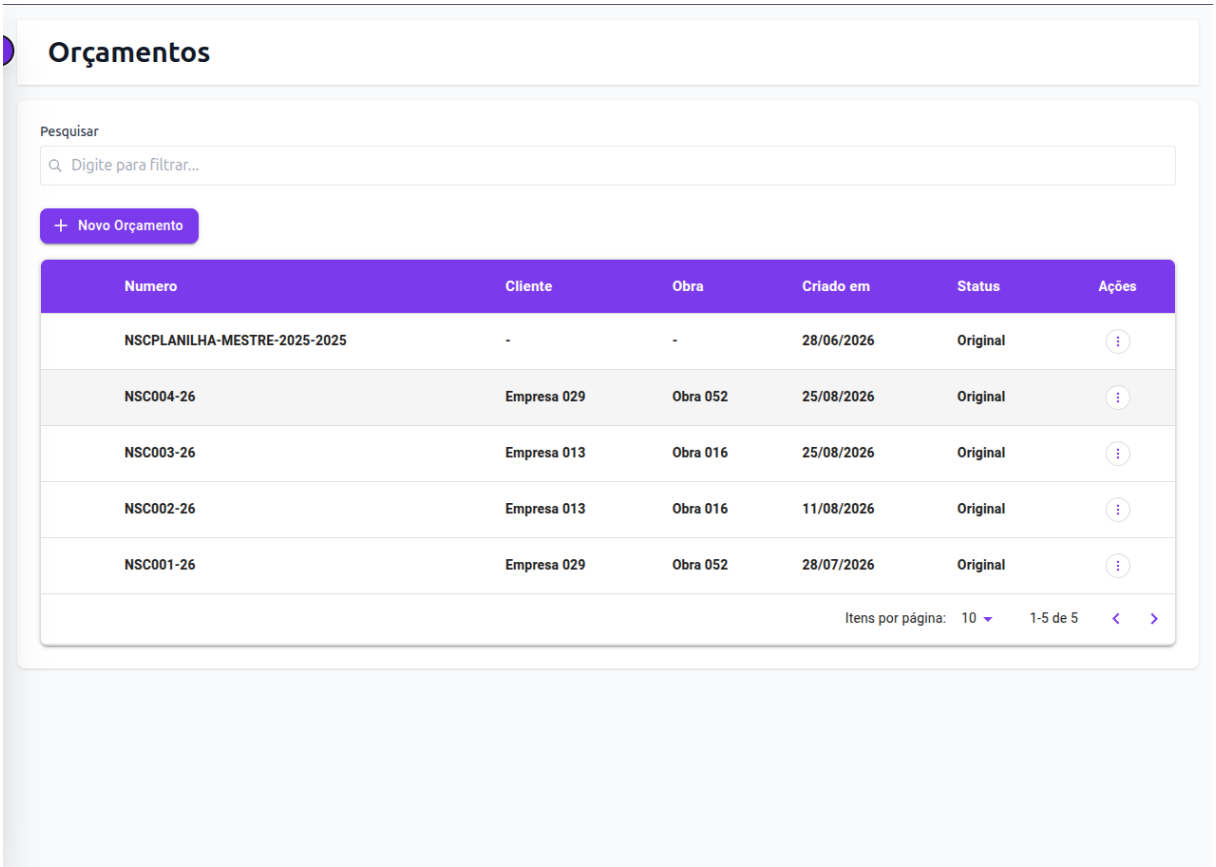
Por fim, o sistema implementa um modelo de controle de acesso baseado em permissões, associando cada usuário a papéis que determinam quais funcionalidades e dados são acessíveis. As permissões são gerenciáveis, permitindo que administradores designem papéis conforme a estrutura organizacional e responsabilidades de cada operador. Todas as operações sensíveis (criação, alteração, remoção de orçamentos e escopos) são registradas em logs de auditoria, permitindo rastreamento completo de quem fez o quê e quando.

2.4 Resultados

2.4.1 Interface do Usuário

A interface do módulo foi concebida para organizar o processo de orçamentação em etapas sequenciais, reduzindo complexidade cognitiva e facilitando o trabalho do usuário. A Figura 4 demonstra a tela inicial, que reúne as operações principais (pesquisa, filtros, criação), enquanto a edição é realizada por meio de abas temáticas.

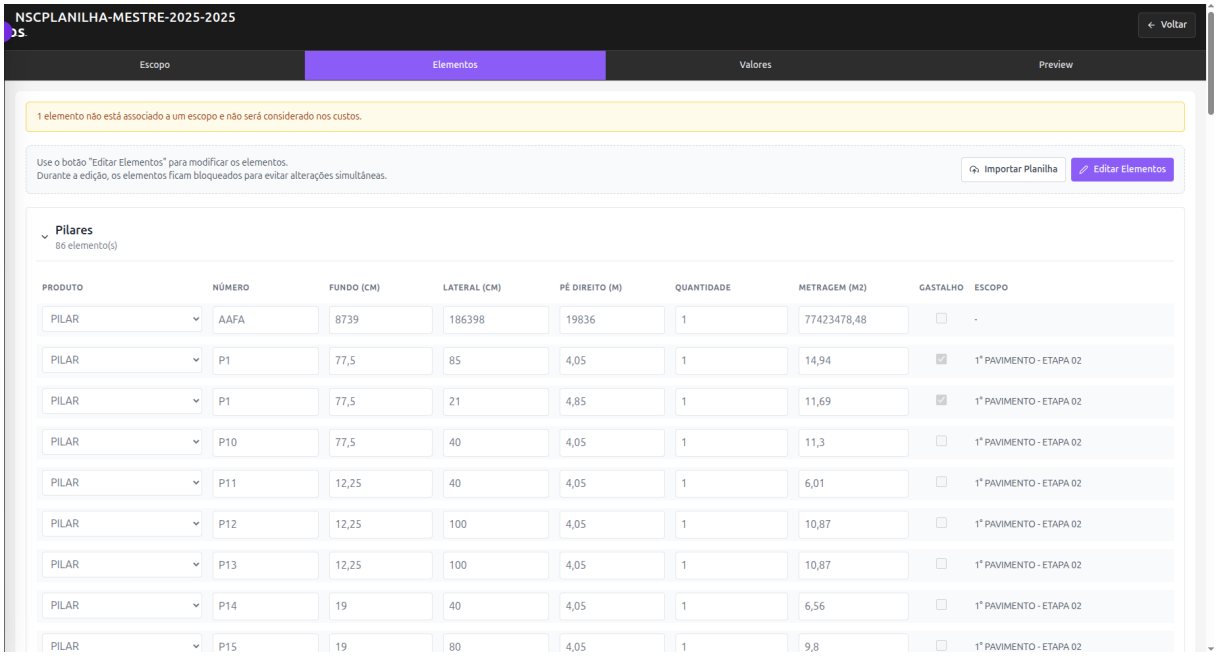
Figura 4 – Interface – Listagem de Orçamentos



Fonte: Elaborado pelos autores, 2026.

A Figura 5 demonstra a aba Elementos, que permite o gerenciamento da lista de elementos do orçamento, com suporte a importação por arquivo e edição manual em lote. Sua interface inclui três seções para cada categoria de elemento onde os itens são mostrados em um formato de tabela, incluindo apenas os campos válidos para seu tipo.

Figura 5 – Interface – Gerenciamento de Elementos



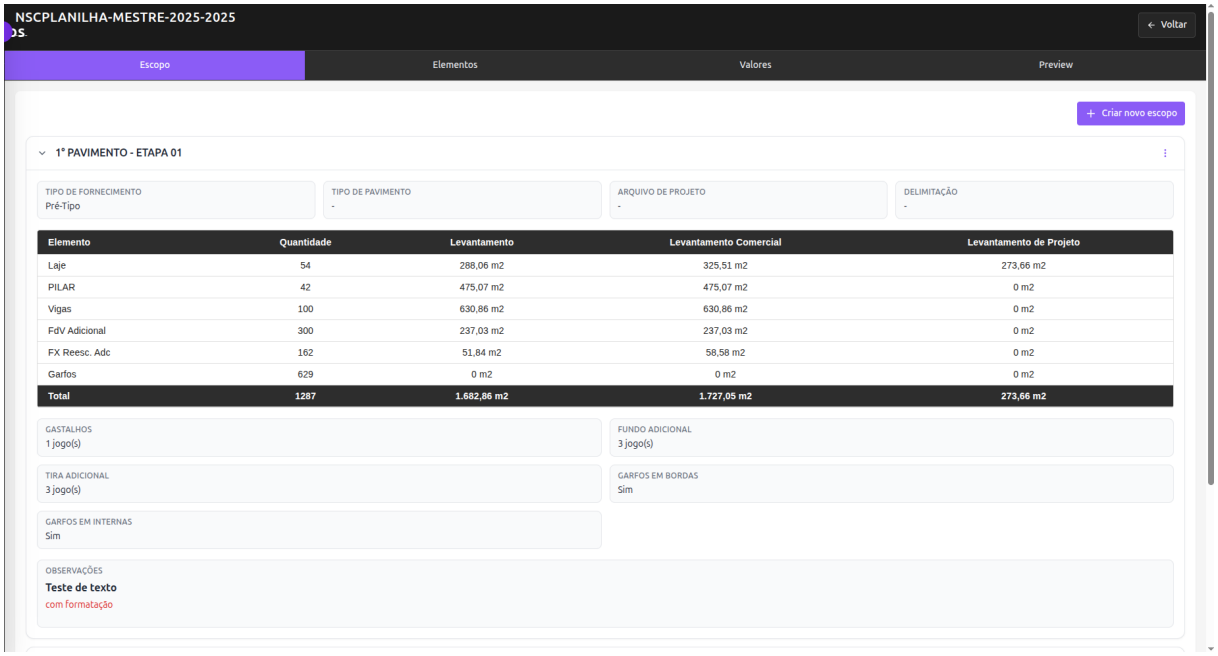
Fonte: Elaborado pelos autores, 2026.

A utilização de seções colapsáveis permite ao usuário ver apenas o que precisa, evitando a necessidade de rolar a tela por grandes distâncias, principalmente quando essa seção pode incluir centenas de itens em cada categoria. Esse elemento de *design* é muito utilizado na criação desse sistema.

O botão “Editar Elementos” inclui uma explicação sobre o bloqueio de edição da seção para outros usuários durante o uso. Essa inclusão é importante para garantir que o usuário entenda a necessidade de limitar o tempo de uso e principalmente para que não confunda a impossibilidade de editar os elementos com um erro no sistema e possua as informações necessárias para liberar seu acesso.

A Figura 6 demonstra a aba Escopos, que separa os projetos em partes menores, permitindo a delimitação de grupos de elementos que podem possuir configurações diferentes. Cada Escopo inclui um resumo de suas informações, incluindo tanto os itens simples, como nomes de responsáveis, quanto uma tabela que resume os elementos inclusos. Isso permite ao usuário extrair uma grande quantidade de informação sem a necessidade de analisar as minúcias do Escopo.

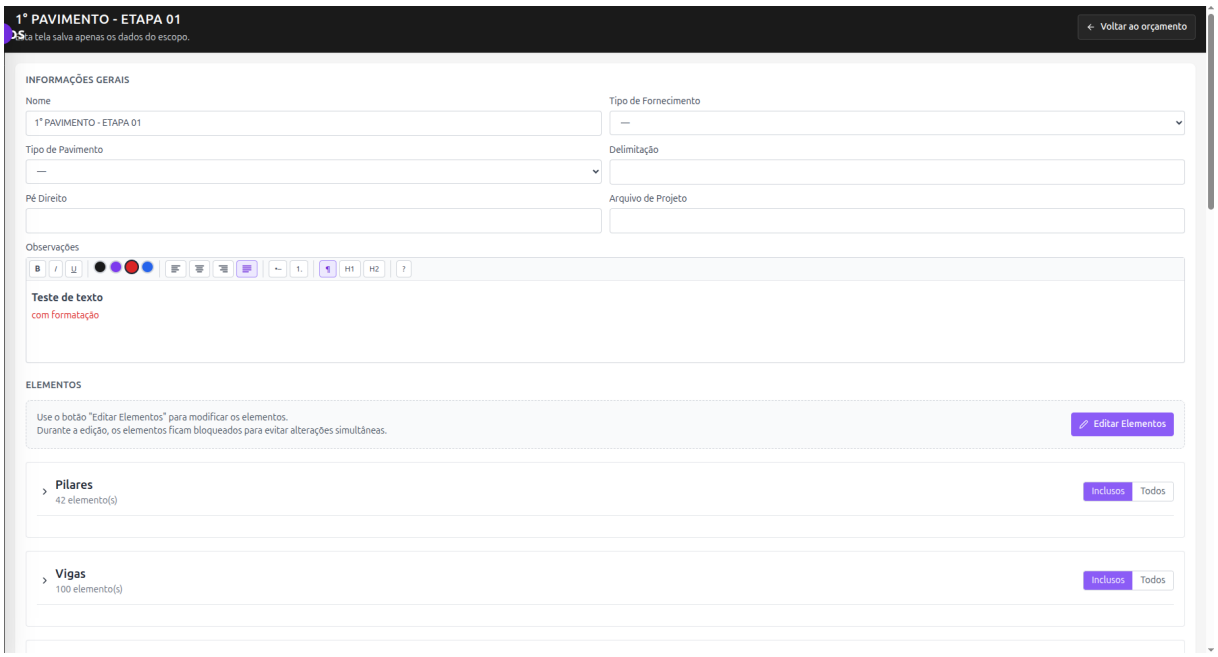
Figura 6 – Interface – Aba de listagem de Escopos



Fonte: Elaborado pelos autores, 2026.

A Figura 7 demonstra a primeira parte da página de edição dos Escopos, que agrupa o gerenciamento dos elementos e a associação de parâmetros específicos (materiais, custos fixos, taxas de uso).

Figura 7 – Interface – Gerenciamento de Escopo 1

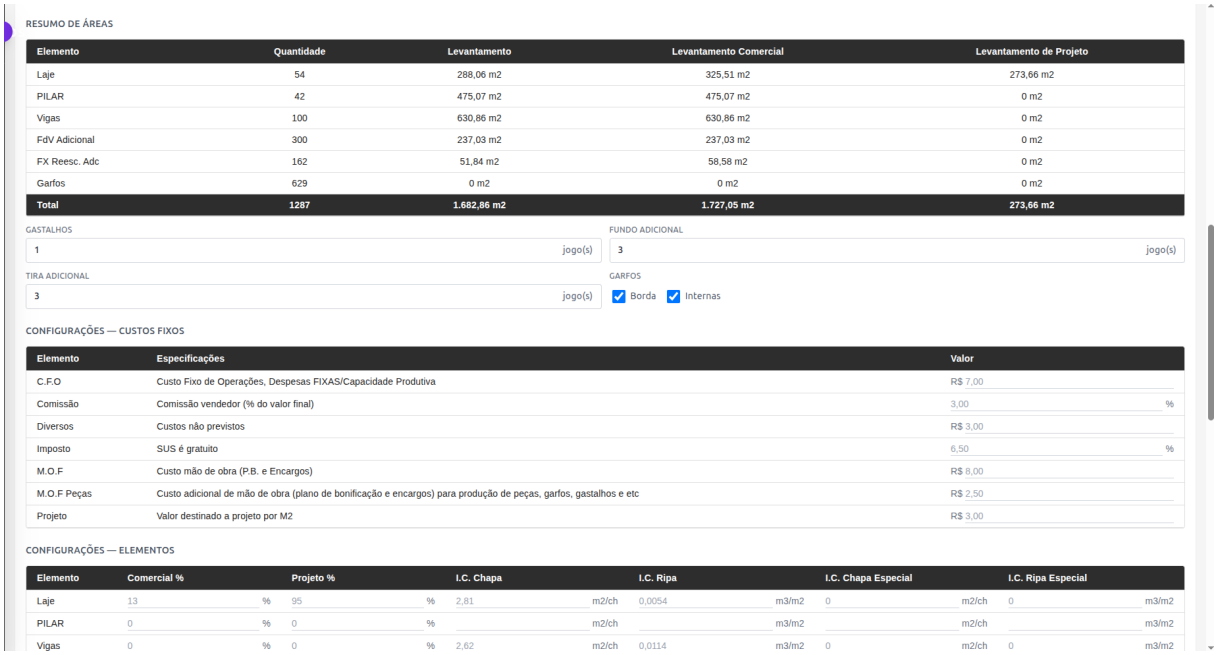


Fonte: Elaborado pelos autores, 2026.

A Figura 8 apresenta a continuação dessa página, cujas configurações são separadas

em múltiplas seções e tabelas. Essa separação limita a quantidade de informação com que o usuário precisa lidar em um determinado momento, facilitando o uso da aplicação.

Figura 8 – Interface – Gerenciamento de Escopo 2



RESUMO DE ÁREAS

Elemento	Quantidade	Levantamento	Levantamento Comercial	Levantamento de Projeto
Laje	54	288,06 m2	325,51 m2	273,66 m2
PILAR	42	475,07 m2	475,07 m2	0 m2
Vigas	100	630,86 m2	630,86 m2	0 m2
FdV Adicional	300	237,03 m2	237,03 m2	0 m2
FX Reesc. Adc	162	51,84 m2	58,58 m2	0 m2
Garfos	629	0 m2	0 m2	0 m2
Total	**1287**	**1.682,86 m2**	**1.727,05 m2**	**273,66 m2**

GASTALHOS: 1 jogo(s) | FUNDO ADICIONAL: 3 jogo(s)

TIRA ADICIONAL: 3 jogo(s) | GARFOS: Borda, Internas

CONFIGURAÇÕES — CUSTOS FIXOS

Elemento	Especificações	Valor
C.F.O	Custo Fixo de Operações, Despesas FIXAS/Capacidade Produtiva	R$ 7,00
Comissão	Comissão vendedor (% do valor final)	3,00 %
Diversos	Custos não previstos	R$ 3,00
Imposto	SUS é gratuito	6,50 %
M.O.F	Custo mão de obra (P.B. e Encargos)	R$ 8,00
M.O.F Peças	Custo adicional de mão de obra (plano de bonificação e encargos) para produção de peças, garfos, gastalhos e etc	R$ 2,50
Projeto	Valor destinado a projeto por M2	R$ 3,00

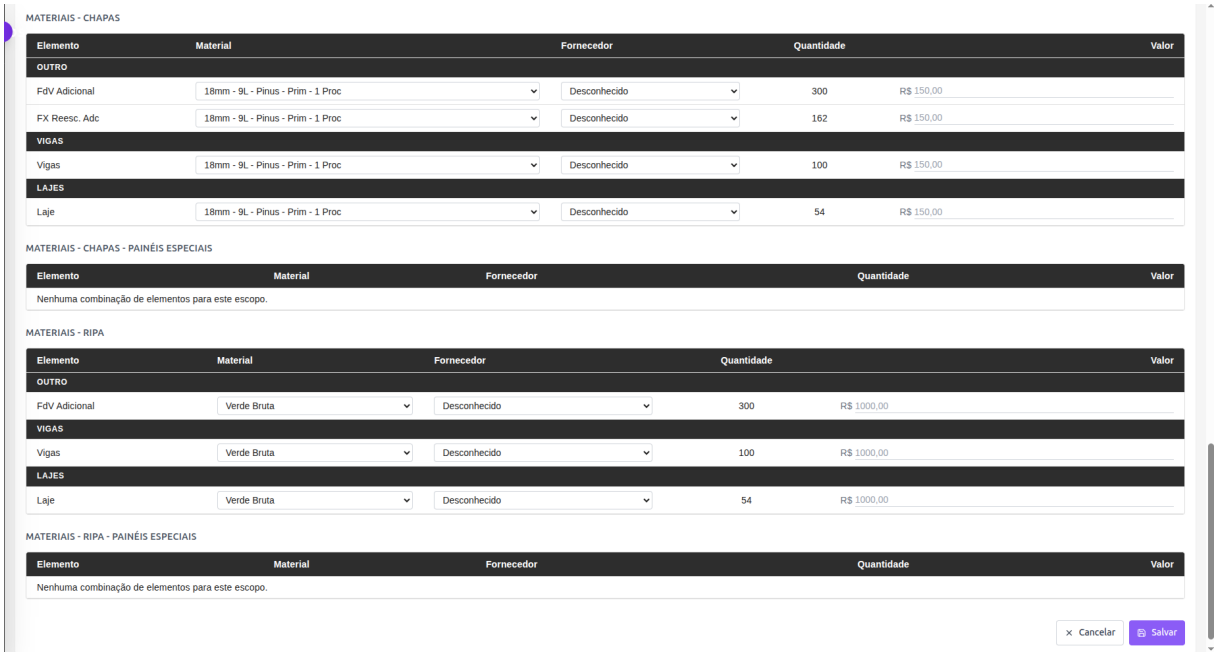
CONFIGURAÇÕES — ELEMENTOS

Elemento	Comercial %	Projeto %	I.C. Chapa	I.C. Ripa	I.C. Chapa Especial	I.C. Ripa Especial
Laje	13 %	95 %	2,81 m2/ch	0,0054 m3/m2	0 m2/ch	0 m3/m2
PILAR	0 %	0 %	m2/ch	m3/m2	m2/ch	m3/m2
Vigas	0 %	0 %	2,62 m2/ch	0,0114 m3/m2	0 m2/ch	0 m3/m2

Fonte: Elaborado pelos autores, 2026.

A Figura 9 apresenta a parte final da página de edição. Em todas as suas partes, a utilização de termos e de uma estrutura similares aos que estavam presentes nas antigas planilhas da empresa aumenta as chances de uma transição fácil.

Figura 9 – Interface – Gerenciamento de Escopo 3

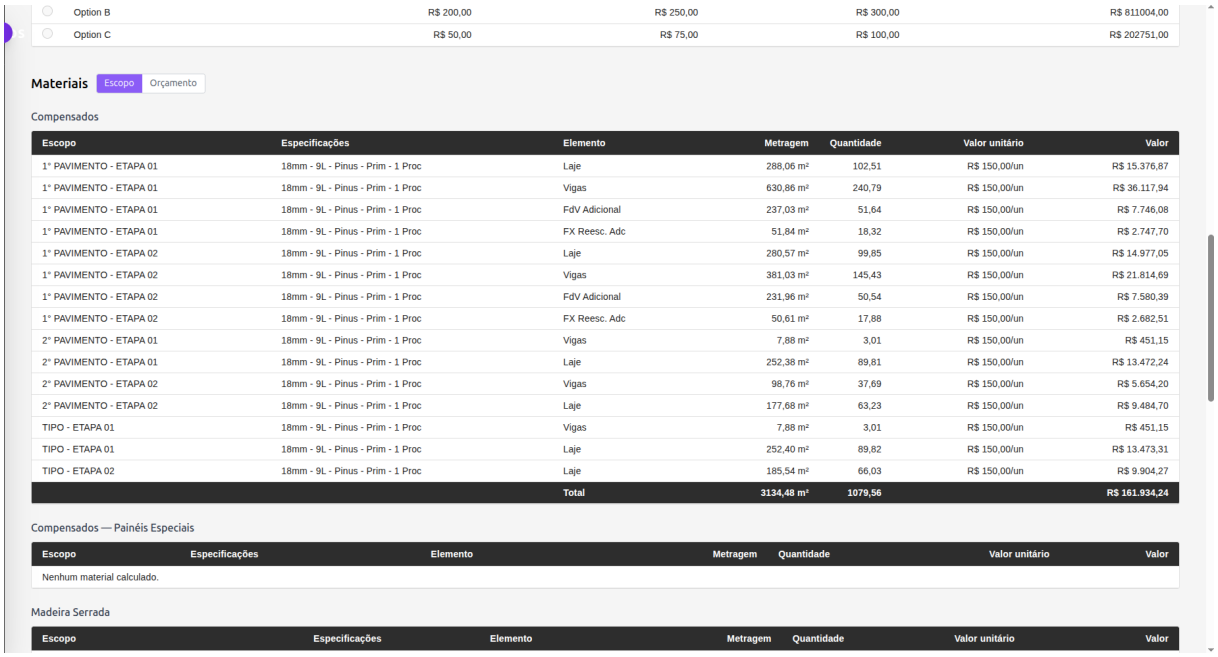


Fonte: Elaborado pelos autores, 2026.

A aba Valores consolida informações de preços, consumo de materiais e custos agregados, facilitando a visualização do impacto financeiro de cada decisão.

A Figura 10 demonstra como todas as configurações dos escopos são agregadas, mostrando os totais de metragem e custo para os materiais.

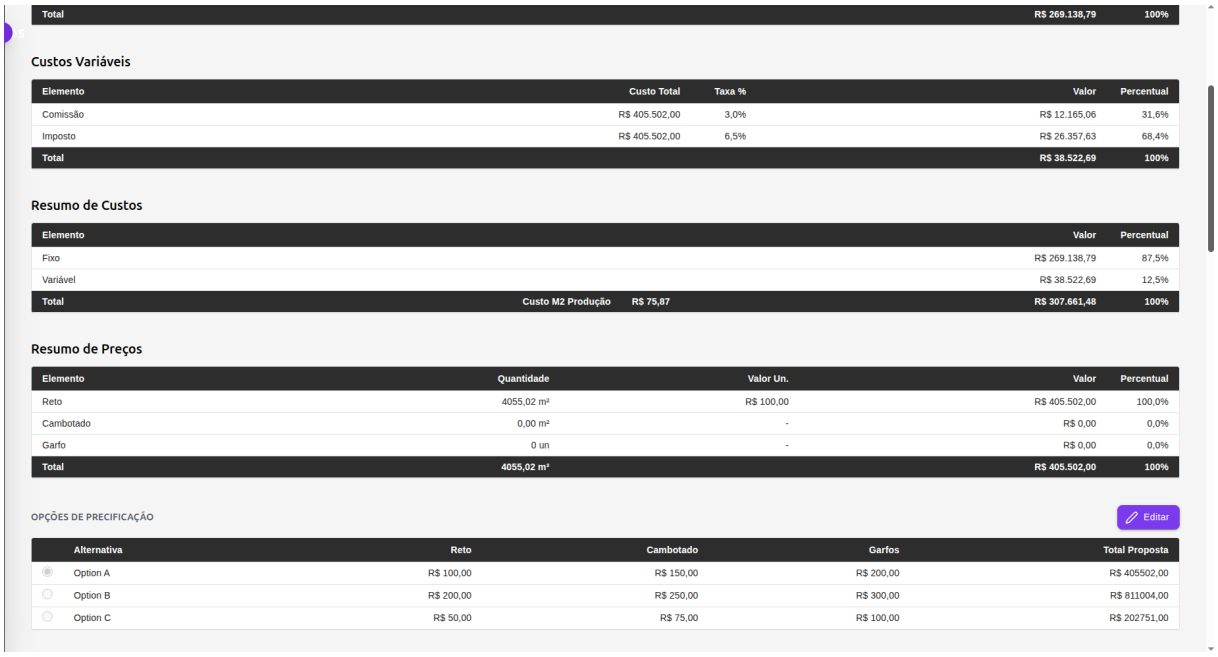
Figura 10 – Interface – Valores – Agregação de Materiais



Fonte: Elaborado pelos autores, 2026.

Esses valores são então somados, o resultado servindo ao usuário como uma referência para a definição do preço a ser cobrado na proposta. A Figura 11 apresenta as opções de precificação, por meio das quais esse preço é configurado.

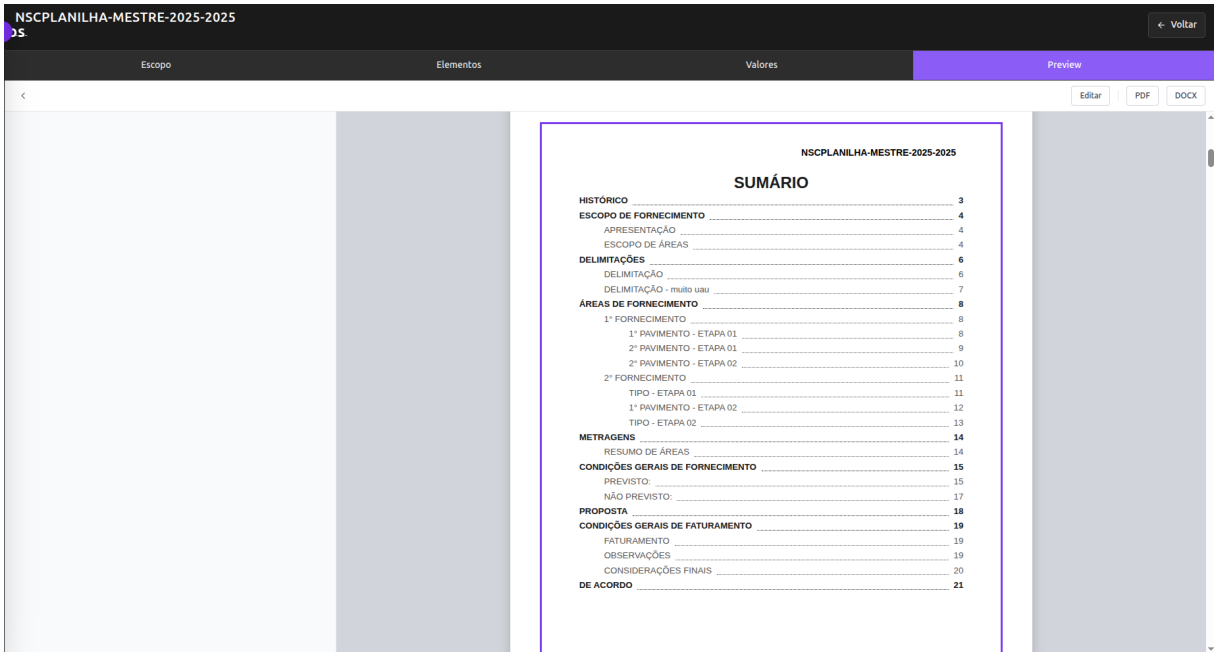
Figura 11 – Interface – Valores – Opções de Precificação



Fonte: Elaborado pelos autores, 2026.

As configurações e inserções de dados são consolidadas na aba *Preview*, que apresenta uma visualização prévia do documento de proposta, permitindo ajuste de texto e formatação antes da geração final. O sistema também permite adicionar imagens. A Figura 12 demonstra o sumário do documento.

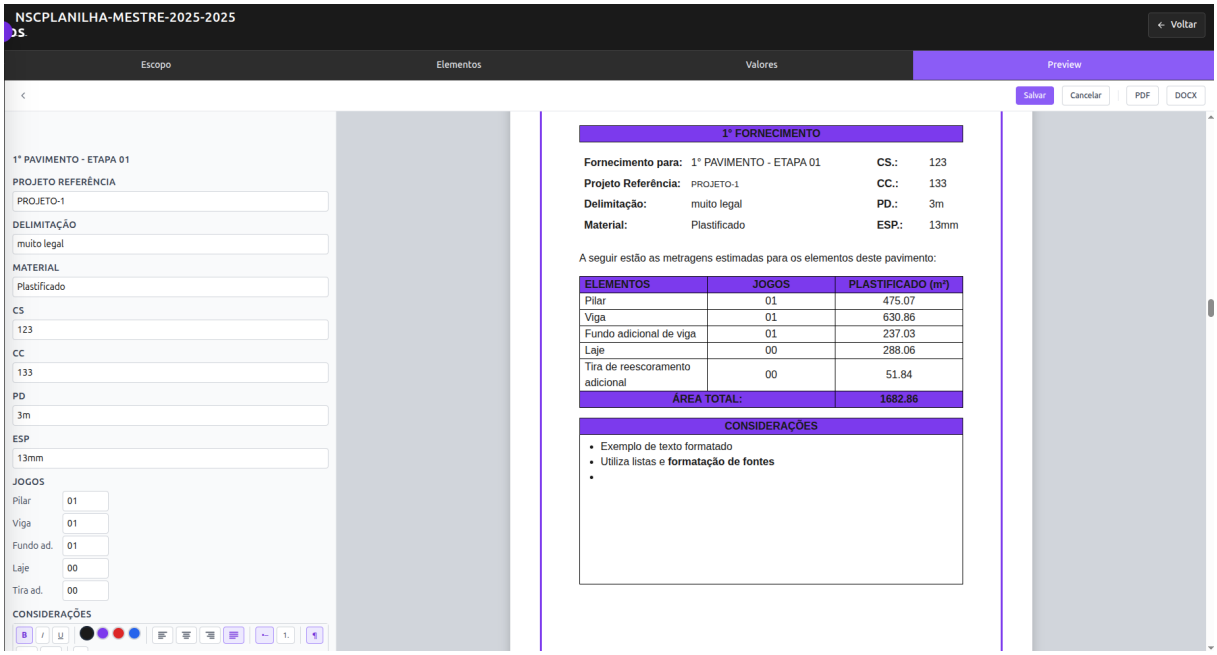
Figura 12 – Interface – Proposta – Sumário do Documento



Fonte: Elaborado pelos autores, 2026.

A Figura 13 apresenta as informações de um Escopo, nas quais o sistema inclui automaticamente o total de metragem.

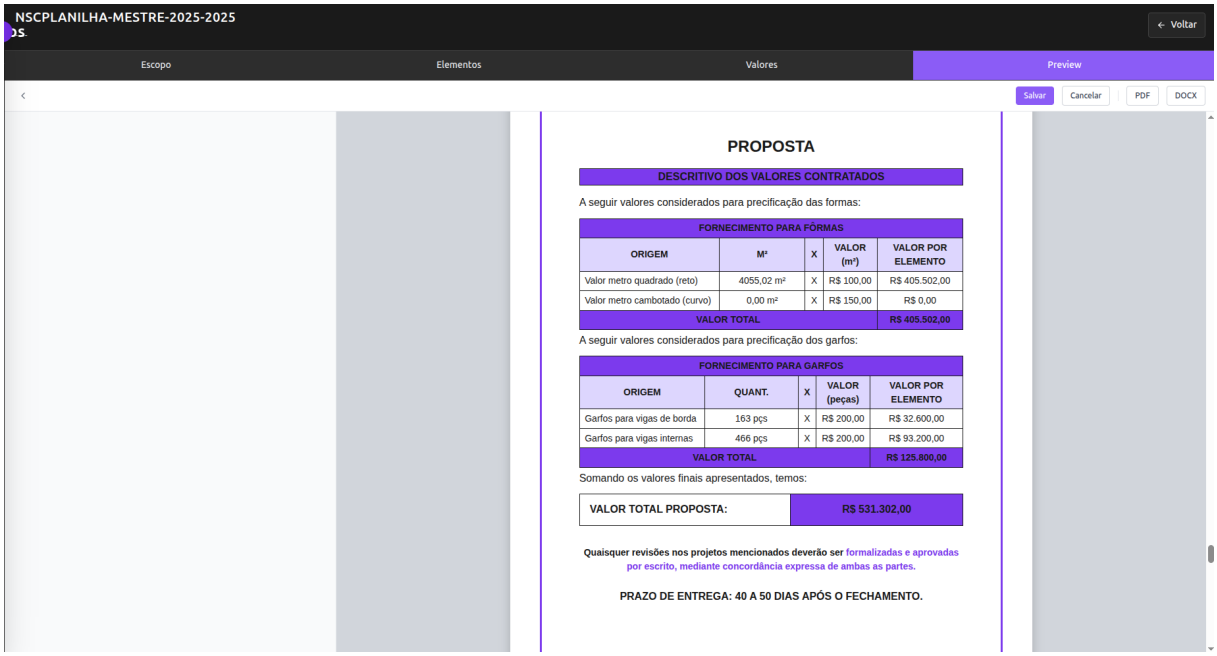
Figura 13 – Interface – Proposta – Informações de um Escopo



Fonte: Elaborado pelos autores, 2026.

A Figura 14 mostra, por fim, os valores da proposta, que incluem os preços finais.

Figura 14 – Interface – Proposta – Valores da Proposta



Fonte: Elaborado pelos autores, 2026.

Após a modificação da proposta, as opções de exportação do documento por PDF e DOCX ficam disponíveis. A primeira realiza a conversão do HTML para o formato diretamente e a segunda conta com um sistema separado que reconstrói os elementos no formato de arquivo DOCX, já que uma conversão direta não é possível.

Cada etapa do fluxo foi desenhada para fornecer sinais claros ao usuário sobre o estado do documento (bloqueios, alterações pendentes) e para permitir ações atômicas e reversíveis quando necessário, buscando um produto que preza pela usabilidade, com o emprego, por exemplo, das dez heurísticas de Nielsen (NIELSEN, 1994).

Todos os objetivos estabelecidos para o projeto foram atingidos e o trabalho realizado produziu uma base sólida e funcional para que a implementação de trabalhos futuros seja realizada.

O mecanismo de coordenação de uso por meio de bloqueio pessimista aplicado por área de edição foi implementado com sucesso, fornecendo retorno visual na interface e proteções para evitar bloqueios persistentes em caso de encerramentos inesperados. Essa funcionalidade poderá ser reutilizada em implementações futuras.

A criação de uma biblioteca contendo as regras de domínio e utilitários de negócio garante consistência entre *frontend* e *backend* e facilita testes automatizados sobre a lógica de cálculos e transformações de dados. Ela poderá ser expandida no futuro, servindo como um local estável para manter as partes mais importantes do sistema, garantindo segurança ao desenvolvedor.

3 CONCLUSÃO

O módulo de orçamento foi concluído, atingindo os objetivos estabelecidos: centralização de dados, organização do fluxo em unidades de trabalho controladas com mecanismos de coordenação, mecanismos de gerenciamento de dados, cálculo de totais e geração do documento final. A arquitetura adotada demonstrou-se adequada para proporcionar coerência lógica e agilidade de desenvolvimento.

Os benefícios práticos observados incluem consolidação de processos anteriormente dispersos, maior segurança em edições colaborativas por meio de bloqueios coordenados, e preservação de histórico de revisões, além da automação dos processos. Estes fatores contribuem diretamente para redução de retrabalho e melhoria da rastreabilidade.

Durante o desenvolvimento, um dos desafios principais identificados foi a compreensão do contexto do cliente, já que o domínio de orçamentação na empresa apresenta complexidades que vão além de processos padrão de engenharia. Compreender plenamente o contexto, as exceções às regras e as necessidades específicas da operação demandou sessões iterativas com o cliente, enfatizando a importância de validação contínua.

Para atender às necessidades desse processo, o sistema foi projetado para ser altamente flexível, permitindo múltiplas exceções. Isso gerou desafios técnicos na modelagem de dados e lógica de validação. A solução adotada, baseada na definição de mecanismos de sobrescrita de valores para determinados escopos de uso, demonstrou ser eficaz, mas gera grande complexidade e compromete a manutenibilidade do projeto.

A resolução de problemas, tanto de implementação quanto decorrentes de mudanças solicitadas pelos usuários, é natural e deve ser conduzida de forma contínua. Quanto a trabalhos futuros, o principal seria a modificação do sistema para permitir que seja utilizado por mais de uma empresa em um modelo *multitenant*.

REFERÊNCIAS

CHEN, P. P.-S. The entity-relationship model: toward a unified view of data. **ACM Transactions on Database Systems**, New York, v. 1, n. 1, p. 9–36, mar. 1976. Disponível em: <https://dl.acm.org/doi/10.1145/320434.320440>. Acesso em: 20 maio 2026. Citado na página 11.

Docker, Inc. **Docker Compose: documentação oficial**. [S.l.]: Docker, Inc., s.d. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 20 maio 2026. Citado na página 13.

Docker, Inc. **Docker: documentação oficial**. [S.l.]: Docker, Inc., s.d. Disponível em: <https://docs.docker.com/>. Acesso em: 20 maio 2026. Citado na página 13.

ExpressJS. **Express: documentação oficial**. OpenJS Foundation, s.d. Disponível em: <https://expressjs.com/>. Acesso em: 20 maio 2026. Citado na página 11.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. Tese (Tese (Doutorado em Informática)) — University of California, Irvine, Irvine, 2000. Disponível em: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 20 maio 2026. Citado na página 11.

FOWLER, M. **BlueGreenDeployment**. martinfowler.com, 2010. Disponível em: <https://martinfowler.com/bliki/BlueGreenDeployment.html>. Acesso em: 20 maio 2026. Citado na página 13.

Meta Platforms, Inc. **React: documentação oficial**. Meta Platforms, Inc., s.d. Disponível em: <https://react.dev/>. Acesso em: 20 maio 2026. Citado na página 11.

Microsoft. **TypeScript: documentação oficial**. Microsoft Corporation, s.d. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 20 maio 2026. Citado na página 11.

NIELSEN, J. **10 usability heuristics for user interface design**. Fremont: Nielsen Norman Group, 1994. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: 20 maio 2026. Citado na página 21.

Node.js Foundation. **Node.js: documentação oficial**. OpenJS Foundation, s.d. Disponível em: <https://nodejs.org/en/docs/>. Acesso em: 20 maio 2026. Citado na página 11.

PostgreSQL Global Development Group. **PostgreSQL: documentação oficial**. s.d. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 20 maio 2026. Citado na página 11.

GLOSSÁRIO

API – Application Programming Interface. Interface que permite a comunicação entre dois sistemas de software.

Backend – Camada de lógica e dados de um sistema, responsável pelo processamento e armazenamento das informações, sem interação direta com o usuário.

Bloqueio Pessimista – Estratégia de controle de concorrência que reserva exclusividade sobre um recurso antes de modificá-lo, prevenindo alterações simultâneas por outros usuários.

CI/CD – Continuous Integration/Continuous Delivery (Integração e Entrega Contínuas). Conjunto de práticas que automatizam a integração, teste e entrega de software ao longo do desenvolvimento.

Docker – Plataforma de conteinerização que permite empacotar aplicações e suas dependências em ambientes isolados e reproduzíveis chamados contêineres.

Docker Compose – Ferramenta complementar ao Docker que permite definir e orquestrar múltiplos contêineres com uma única configuração, simplificando a criação de ambientes complexos.

Express.js – Framework web para Node.js utilizado para a construção de APIs e servidores HTTP.

Frontend – Camada de apresentação de um sistema, responsável pela interface visual e pela interação direta com o usuário.

Modal – Elemento de interface que exibe conteúdo em sobreposição à tela principal, bloqueando a interação com o restante da aplicação até ser fechado ou confirmado.

Multitenant – Arquitetura de software que permite que múltiplos clientes (*tenants*) compartilhem a mesma instância de um sistema, mantendo isolamento de dados e configurações entre eles.

Node.js – Ambiente de execução JavaScript no lado do servidor, utilizado para construção de APIs e aplicações de rede.

REST – Representational State Transfer. Estilo arquitetural para APIs que define restrições sobre como os serviços devem se comunicar via HTTP.

Transação – Operação indivisível executada completamente ou revertida em sua totalidade (tudo ou nada), garantindo a integridade dos dados.

TypeScript – Linguagem de programação baseada em JavaScript que adiciona tipagem ao código, aumentando a segurança e a previsibilidade do desenvolvimento.

Versionamento – Processo de manter e rastrear múltiplas versões de um registro, preservando as relações entre elas para fins de auditoria e rastreabilidade.